

Systems Science & Control Engineering

An Open Access Journal

ISSN: 2164-2583 (Online) Journal homepage: www.tandfonline.com/journals/tssc20

MP-GestLSTM: real time gesture detection using MediaPipe and LSTM

T.S. Varshini & P. Rukmani

To cite this article: T.S. Varshini & P. Rukmani (2025) MP-GestLSTM: real time gesture detection using MediaPipe and LSTM, Systems Science & Control Engineering, 13:1, 2587853, DOI: [10.1080/21642583.2025.2587853](https://doi.org/10.1080/21642583.2025.2587853)

To link to this article: <https://doi.org/10.1080/21642583.2025.2587853>



© 2025 The Author(s). Published by Informa UK Limited, trading as Taylor & Francis Group.



Published online: 18 Nov 2025.



Submit your article to this journal [↗](#)



Article views: 2033



View related articles [↗](#)



View Crossmark data [↗](#)



Citing articles: 5 View citing articles [↗](#)

MP-GestLSTM: real time gesture detection using MediaPipe and LSTM

T.S. Varshini and P. Rukmani

School of Computer Science and Engineering, Vellore Institute of Technology Chennai Campus, Chennai, India

ABSTRACT

Sign language is a crucial form of communication for individuals with hearing impairments; thus, the development of accurate and efficient sign language recognition systems is of significant importance. The main aim of the work is to develop an accurate and efficient sign language recognition model using MediaPipe (MP) and deep learning techniques. MediaPipe (BlazePose) is a top-down model with an encoder-decoder architecture to predict the keypoints for all joints. MediaPipe detects 33 body(pose) landmarks, 21 hand landmarks, and 478 3-dimensional face landmarks. A custom dataset called MP-Gest consists of 800 videos belonging to 20 classes at the word level of American Sign Language (ASL) is generated by two signers. A selection of MediaPipe keypoints is extracted from this custom dataset and used to train a Long Short-Term Memory (LSTM)-based custom model called MP-GestLSTM. The MP-GestLSTM model achieved a validation accuracy of 93.75% and testing accuracy of 94.38%, indicating that the model outperformed existing research with minimal error. The proposed model gives enhanced recognition accuracy using a simple custom architecture and contributes to the advancement of sign language recognition by overcoming the existing limitations.

ARTICLE HISTORY

Received 16 April 2025

Accepted 6 November 2025

KEYWORDS

Sign Language; MediaPipe; LSTM; MP-GestLSTM; Deep Learning; CNN (Convolution Neural Network)

1. Introduction

Sign language recognition is helpful in breaking down communication barriers between deaf and hearing individuals by enabling machines to comprehend and interpret sign-language gestures. Like how spoken language recognition converts speech into text or commands, sign language recognition involves translating manual gestures, facial expressions, and body movements into understandable data for machines. Real-time sign language detection using deep learning has attracted considerable attention. Current methods struggle to achieve a balance between accuracy and speed, especially in dynamic and real-world scenarios. Effective sign-language recognition often requires real-time processing for seamless communication. It's also difficult to achieve low latency recognition, especially in resource-constrained environments, such as mobile devices. Building a universal automatic model for sign language recognition is extremely difficult because of challenges such as inter- and intra-subject variability, illumination conditions, partial occlusions, different viewpoints and resolutions, and background artifacts. Techniques such as CNNs and recurrent neural networks (RNNs) have shown promise in addressing this issue. CNNs effectively process visual input, thus are widely used in analysing images and video frames of sign language gestures and learn to recognize different hand shapes, movements, and orientations. RNNs understand the sequential nature of gestures and movements by capturing the temporal dynamics of sign language, thus making them valuable for tracking the movement of the hands and fingers, which is crucial for real-time sign language detection.

Existing recognition methodologies face challenges in achieving a balance between accuracy and speed. This study aims to address this gap by proposing and implementing a novel real-time sign language detection system that leverages deep learning architectures. A better sign language detection model named MP-GestLSTM is proposed in this study, which is trained using the MP-Gest dataset, which consists of 800 videos of 20 classes (each class with 40 videos). The main contributions of this study are as follows:

- The study has identified important keypoints of body joints, hands, and face that help in the classification of sign language using MediaPipe.

- Proposed 'MP-GestLSTM', an LSTM based model trained with keypoints extracted from videos by leveraging compact, informative spatial and temporal representations of human pose and motion. MP-GestLSTM has achieved high accuracy on the generated 'MP-Gest' dataset comprising 800 videos belonging to 20 classes.
- We developed a real-time application with webcam input to validate our model and to assist those in need.

In the proposed method, keypoints of important body joints that help in recognizing gestures are extracted from each video and are used to train a custom LSTM model. In addition, the keypoints were padded before training to obtain a uniform shape. The model achieved good testing accuracy and validation accuracy.

The primary problem addressed in this work is the accurate and efficient recognition of sign language gestures from video data to facilitate seamless communication for individuals with hearing impairments. Existing models often struggle with limited dataset quality, complex architectures, and poor real-time adaptability. To overcome these challenges, this work focuses on identifying crucial human keypoints (hands, body joints, and facial landmarks) using MediaPipe and leveraging them through a Lightweight LSTM-based model (MP-GestLSTM) trained on a custom-built MP-Gest dataset comprising 800 videos across 20 ASL word-level classes. The proposed approach not only enhances gesture classification accuracy by effectively capturing spatial-temporal patterns but also demonstrates real-time applicability through a webcam-based system designed to support and empower individuals with communication barriers.

The remainder of this paper is organized as follows. Section 2 provides a comprehensive Literature Review, summarizing relevant studies and recent advancements in sign language recognition and gesture detection frameworks. Section 3 describes the Proposed Methodology, detailing the MP-Gest Dataset Creation and Libraries Used (Section 3.1), Data Preprocessing of the MP-Gest Dataset (Section 3.2), and the development of the MP-GestLSTM Model (Section 3.3). Section 4 presents the Results and Discussion, including the Performance Evaluation of the MP-GestLSTM Model (Section 4.1) and the Performance Assessment of the Real-Time Application (Section 4.2). Finally, Section 5 concludes the paper and outlines potential avenues for Future Work in this domain.

2. Literature review

The field of sign language recognition and translation has witnessed significant advancements in recent years. Most of these were driven by the intersection of deep learning techniques and datasets specifically tailored for sign language analysis. This literature review provides an overview of several recent studies that have contributed to the development of sign language recognition systems using various approaches, datasets, and architectures. Some of the diverse datasets, such as MINDS-Libras, Indian Sign Language (ISL), and American Sign Language (ASL) were trained with novel methodologies like hybrid deep transfer learning and semantic translation systems. These have collectively contributed to improving the accuracy, robustness, and practicality of sign language translation technology. A comparison with the existing literature is presented in Table 1.

De Castro et al. (2023) leveraged DepthMaps alongside RGB frames. Their utilization of the HHDS (hands, heads, distance, and speed maps) multi-stream 3D CNN proved to be efficient and remarkably accurate. Notably, the HHDS architecture, upon integrating RGB and depth maps, showed a marginally higher performance than its precursor. This pioneering work has proposed the integration of encoded language and multi-stream architecture and has also introduced new possibilities in sign language detection. The datasets employed in their study, including MINDS-Libras, INCLUDE-50, and LIBRAS-UFOP, proved the diversity of their approach, providing a robust foundation for further advancements in the field.

Integrated approaches of machine-learning solutions for real-world applications have enhanced the accessibility of sign language recognition. Sharma and Singh (2023) introduced an integrated approach that combines the MobileNetv2 architecture with the transfer-learning techniques. This fusion yielded impressive results, resulting in high accuracy rates. Notably, the framework they proposed succeeded in reducing the number of parameters, signalling a significant decrease in computational complexity. This reduction is particularly vital for real-time applications, in which the computational requirements must be minimized.

Sharma and Singh developed a real-time prototype to underscore the practical applicability of their approach. The compact architecture of the model not only meets the demands of real-time processing but also delivers exceptional precision. The utilization of datasets such as the Indian Sign Language (ISL) with 28

Table 1. Comparison of existing literature.

References	MINDS-Libras dataset	Classes	Type	Methodology	Performance (Accuracy)
De Castro et al. (2023)	MINDS-Libras dataset	20	videos	HHDS (hands + head + distance and speed maps) Multi-stream 3D CNN	0.90±0.07
De Castro et al. (2023)	INCLUDE-50	20	videos	RGB + DM (Depth Maps) HHDS	0.91±0.07
De Castro et al. (2023)	Korean Sign Language	50	videos	RGB + DM HHDS	0.94
De Castro et al. (2023)	LIBRAS-UFOP dataset	77	videos	RGB + DM HHDS	0.50±0.05
De Castro et al. (2023)	Own Dataset – ISL	56	videos	RGB + DM HHDS	0.74±0.04
Sharma and Singh (2023)	ASL – public	28 classes	images	Mobilenetv2 integrated with transfer learning	99.30%
Sharma and Singh (2023)	Own dataset	10 classes	images	Mobilenetv2 integrated with transfer learning	99.90%
Sreemathy et al. (2023)	Own dataset	80 classes	images	You Only Look Once version 4 (YOLOv4)	98.80%
Sreemathy et al. (2023)	Bangla Sign Language for alphabets – ‘Ishara-Lipi’	80 classes	images	Support Vector Machine (SVM) with media-pipe	98.62%
Sunanda Das et al. (2023)	Bangla Sign Language for numerals – ‘Ishara Bochon’	36 Bengali sign characters (6 vowels and 30 consonants)	images	Deep transfer learning-based convolutional neural network with a random forest classifier – Hybrid (VGG16 + RF)	91.67%
Sunanda Das et al. (2023)	Own Dataset	10 classes	images	Deep transfer learning-based convolutional neural network with a random forest classifier – Hybrid (VGG16 + RF)	97.33%
Stellin et al. (2022)	WLASL	10 classes	images	Convolution Neural Networks (CNN) based model	99.13%
Du et al. (2022)		100 classes	videos	Masking future operation with full self-attention ViT framework	Top1 – 80.72%
Du et al. (2022)	WLASL	300 classes	videos	Masking future operation with full self-attention ViT framework	Top1 – 70.14%
Du et al. (2022)	WLASL	1000 classes	videos	Masking future operation with full self-attention ViT framework	Top1 – 57.13%
Du et al. (2022)	NMFs-CSL	1067 classes	videos	Masking future operation with full self-attention ViT framework	Top1 – 70.1%
Kasapbaşı et al. (2022)	Own dataset – American Sign Language alphabet	26 classes	images	Convolutional Neural Network (CNN)	99.38%
Sharma and Singh (2021)	Own dataset – Indian sign language (ISL) gestures consisting of 2150 images	26 ISL alphabets classes	images	Deep learning based convolutional neural network (CNN) model	94.83%
Sharma and Singh (2021)	Own dataset – Indian sign language (ISL) gestures consisting of 2150 images	17 ISL words classes	images	Deep learning based convolutional neural network (CNN) model	99.96%
Sharma and Singh (2021)	American sign language (ASL) dataset	10 classes	images	Deep learning based convolutional neural network (CNN) model	100%
Sign Language Translation Using Deep Convolutional Neural Networks (2020)	‘Kaggle’ American Sign Language fingerspelling dataset	24 classes	images	Integration of single-shot multibox detection (SSD), Inception 3 and SVM	99.90%
Elsayed and Fathy (2020)	pre-made Arabic sign language dataset(ArSL2018)	32 class	images	Semantic Deep Learning – Deep Convolution Neural Network (CNN)	88.87%
(Elsayed and Fathy (2020)	own dataset – words	10 classes	images	Semantic Deep Learning – Deep Convolution Neural Network (CNN)	94.31%
Tolentino et al. (2019)	ASL Alphabet Dataset	26 classes	images	Convolutional Neural Network (CNN)	90.04%
Tolentino et al. (2019)	ASL Number Dataset	10 classes	images	Convolutional Neural Network (CNN)	93.44%
Tolentino et al. (2019)	ASL Static Word Dataset	35 classes	images	Convolutional Neural Network (CNN)	97.52%

classes and the American Sign Language (ASL) with 10 classes further validates the versatility and efficacy of their methodology.

Combining complementary techniques often enhances the performance and reliability of object-detection and classification systems. Sreemathy et al. (2023) presented an innovative exploration of object detection and classification by leveraging a diverse dataset comprising 80 classes of images. Their work adopted a deep learning architecture, You Only Look Once version 4 (YOLOv4) architecture. This was important in building cutting-edge methodologies to achieve robust object detection capabilities.

Furthermore, the authors extended their investigation by exploring the integration of a Support Vector Machine (SVM) with MediaPipe for the same dataset. While the standalone SVM-MediaPipe approach yields a slightly lower architecture performance, it sets the stage for a novel combined model. This combined model harnessed the feature extraction capabilities of MediaPipe alongside the classification capability of SVM and exhibited a higher real-time accuracy. Notably, this integrated approach also addresses the small inaccuracies present in the individual models, emphasizing the synergy achieved through their methodology.

Deep learning and traditional machine learning techniques have always yielded a high accuracy in sign language recognition tasks. Sunanda Das et al. (2023) proposed a hybrid approach tailored for Bangla sign language recognition with a specific emphasis on alphabet and numeral recognition. Their methodology combines a deep transfer learning-based convolutional neural network (CNN), specifically VGG16, with a random forest (RF) classifier, termed hybrid (VGG16 + RF). Das et al. employed a grid search algorithm to optimize models across various batch sizes for robust performance. In addition, the implemented mini-batch technique introduced regularization effects and mitigated generalization errors, thereby enhancing the reliability of the model. Although VGG16 offers powerful feature extraction capabilities, it can be computationally expensive and may suffer from overfitting owing to its large number of parameters.

Similarly, although RF classifiers are known for their simplicity and efficiency, they often struggle with high-dimensional feature spaces and lack the ability to capture complex relationships within data. The authors' hybrid approach effectively mitigates these limitations, offering a promising solution for Bangla sign language recognition and paving the way for future advancements in this field.

Effective gesture collection is easier with preprocessing techniques such as image resizing, noise or background removal, and edge identification. Stellin et al. (2022) introduced the HandGCNN model, specifically tailored for gesture recognition, to enhance voice-assistance systems. Their model was trained on a dataset comprising ten classes of images, enabling high accuracy in recognizing gestures. This accuracy is pivotal for seamless integration with voice assistance technology, promising an enhanced user experience and improved accessibility in interactive systems.

While convolutional neural networks (CNNs), such as HandGCNN, excel in capturing spatial patterns and features within images, enabling robust gesture recognition, they may require substantial computational resources for training and inference. Additionally, CNNs are susceptible to overfitting, particularly when dealing with limited training data or complex gesture variations. Stellin et al.'s innovative model addressed these challenges by leveraging CNNs and effectively interpreting gestures to bridge the gap between visual- and voice-based interactions in assistive technology.

Transformers capture long range dependencies and contextual information with their attention mechanism making them well suited for sequence modelling tasks such as sign language recognition. Additionally, the Vision Transformer (ViT) framework enables the holistic processing of image sequences, enhancing the model's ability to recognize complex gestures. Du et al. (2022) presented an innovative approach to word-level sign language recognition by leveraging a full transformer network with masking future operations within a ViT framework. They introduced masking future transformer which significantly enhanced frame representation by utilizing spatial encoders to clean the dataset images and extract features. Moreover, their temporal module implemented transformers to interpret sequences, facilitating an effective understanding of sign-language gestures over time.

This methodology marks a significant departure from traditional convolutional neural network (CNN) architectures, offering a promising alternative for sign-language recognition tasks. However, transformers may suffer from high computational overhead, particularly when dealing with large datasets or long sequences. Furthermore, training transformer-based models may require extensive computational resources and data augmentation techniques to prevent overfitting, particularly in scenarios with limited training data. The

authors' innovative approach showcases the potential of transformers in revolutionizing sign language recognition, offering insights into the future of assistive technologies for the deaf and hard-of-hearing communities.

Robust dataset creation methodologies are important for achieving accurate and reliable results in real-world applications. Kasapbaşı et al. (2022) introduced DeepASLR, a Convolutional Neural Network (CNN)-based human-computer interface designed for American Sign Language (ASL) recognition. Their work emphasized the critical role of dataset creation under variable conditions to achieve improved accuracy using deep learning techniques. By focusing on ASL recognition, Kasapbaşı et al. highlighted the potential of CNN-based approaches and the need for effective communication interfaces for individuals with hearing impairments. The utilization of deep learning techniques highlights the importance of leveraging large and diverse datasets to enhance model performance, particularly in complex tasks, such as sign language recognition.

Real-world constraints and usability factors must be addressed when developing assistive technologies for individuals with hearing impairments. Sharma and Singh (2021) presented a vision-based hand gesture recognition system built with deep learning techniques for interpreting both Indian Sign Language (ISL) and American Sign Language (ASL). Their research not only focused on the recognition of gestures but also extended to testing with augmented data, demonstrating robustness against rotation and scaling transformations. By addressing these challenges, Sharma and Singh's approach enhances the adaptability of the system to varying environmental conditions and user interactions. Furthermore, researchers have developed a system that eliminates user dependency and the need for external hardware, thereby showcasing the potential for practical implementation and widespread accessibility.

Sign Language Translation Using Deep Convolutional Neural Networks (2020) investigated sign language translation utilizing deep convolutional neural networks (CNNs), with a focus on the 'Kaggle' American Sign Language fingerspelling dataset for the extraction of intricate features from image data. Their study integrated various techniques, including single-shot multibox detection (SSD), Inception 3, and Support Vector Machine (SVM). This combination offered several advantages.

First, the utilization of deep CNNs, such as Inception 3, enabled the extraction of complex visual features from sign language gestures, which aided the model in accurately recognizing and interpreting them. Second, the incorporation of SSD facilitated efficient object detection and improved the computational efficiency. Finally, the integration of SVM provides a robust classification mechanism, which is particularly useful for handling subtle nuances and variations in sign gestures. These techniques created a comprehensive and effective solution for sign language translation and promising advancements in accessibility and communication for individuals with hearing impairment. The authors' approach demonstrated the potential of integrated deep learning techniques for innovative assistive technologies for deaf and hard-of-hearing communities.

The integration of deep learning and ontology methods represents a significant step forward in advancing Semantic Deep Learning for sign language interpretation and fostering inclusive communication for the deaf and hard-of-hearing community. Elsayed and Fathy (2020) introduced an innovative approach that combines deep learning and ontology methods, specifically tailored for Semantic Deep Learning, with a focus on the Arabic Sign Language dataset (ArSL2018). Their methodology begins with comprehensive pre-processing operations aimed at enhancing the quality and uniformity of the dataset. These operations include cropping to isolate the hand object, resizing to a common resolution of 64×64 pixels, and converting images to grayscale to simplify the processing. Additionally, the dataset images underwent smoothing using Gaussian filters and a median blur to reduce noise, compensating for variations in environmental conditions during image capture. Notably, the dataset comprises real images captured using a mobile camera in uncontrolled settings, which introduces challenges such as differences in lighting, skin colour, and environmental factors. To address semantic interpretation, Elsayed and Fathy proposed a multi-sign language ontology (MSLO), which leverages linguistic ontology principles drawn from WordNet and multilingual WordNet in the sign language domain. They employed SPARQL to semantically predict corresponding meanings and sign gestures across different languages, offering a promising avenue for cross-linguistic sign language translation and communication accessibility.

Hand Skin Colour Detection using image-processing techniques enhances the robustness and accuracy of the system by effectively isolating and analysing hand gestures against varying backgrounds and skin tones. Tolentino et al. (2019) and He (2019) presented pioneering studies in sign language recognition utilizing Convolutional Neural Networks (CNNs) across three distinct datasets, each focusing on different aspects of American Sign Language (ASL).

Using a Faster R-CNN, a hand-locating network can be created to identify the hand portion in an image or sign language video. Long- and short-term memory (LSTM) and a 3D CNN feature extraction network can be implemented with Faster R-CNN to solve the problem of RGB sign language images (He, 2019). Semi-supervised tokenization using Adversarial, Multitask, Transfer Learning with 3D-CNN and 2D-CNN models has proven to be efficient at recognizing the hand shape and pose of humans (Orbay & Akarun, 2020). Translating monolingual text to its gloss sequence has helped address the limited quantity of parallel sign-text data. Together with encoder-decoder frameworks, this system efficiently addresses the research problem (Zhou et al., 2021). The combination of OpenPose for key point estimation with a multi-head attention mechanism in transformers can effectively recognize the signs. This feature learning methodology was successfully implemented using the Flemish dataset by De Coster et al. (2020). A normalization approach for the signing space of human hands and human pose independent transformer network has been efficient in improving the recognition systems (Boháček & Hrz, 2022). Positional embeddings, self-attention layers, and multilayer perceptron networks in transformer networks efficiently classify sign gestures within a very short training period (Kothadiya et al., 2023). One of the standout features of the research on sign language recognition systems is the system's remarkable capability to generate words through fingerspelling, without the need for sensors or other external technologies. This innovation is a significant advancement in accessibility and communication for individuals with hearing impairments and offers a seamless and intuitive method of expressing words through sign language. CNNs in sign language recognition underscore the importance of integrating image-processing techniques to address real-world challenges and improve the overall efficacy of assistive technologies for the deaf and hard-of-hearing communities.

Svendsen and Kadry (2024) developed a dataset for Norwegian Sign Language (NSL) recognition, consisting of 24,300 images belonging to 27 letters. The data were collected from a male and a female signer under varying lighting, background, and angle conditions. This work addresses the lack of NSL datasets and provides a strong foundation for developing and evaluating machine learning-based sign recognition models, supporting future research in automated NSL communication systems.

Existing datasets, such as WLASL (Li et al., 2020), hinder the development of accurate sign language recognition systems due to an insufficient number of samples per class, variations in illumination, and inconsistent video resolutions. These limitations complicate reliable keypoint extraction and model generalization. To overcome these challenges, this work introduces the MP-Gest dataset, a custom-collected video-based dataset comprising 800 samples across 20 ASL word-level classes, captured using a webcam from two signers. Unlike existing datasets, MP-Gest ensures controlled recording conditions, consistent framing, and clearer visibility of gestures. The proposed MP-GestLSTM framework utilizes this dataset, MediaPipe-based landmark extraction, and LSTM-based model to enhance recognition accuracy, reduce complexity, and improve robustness under real-time conditions.

3. Proposed methodology

The proposed approach begins with data acquisition and feature extraction, model training, and real-time testing. Initially, videos were collected from signers performing selected gestures mentioned in section 3.1, which were then divided into individual frames for analysis. From each frame, MediaPipe was used to extract keypoints corresponding to the hand, face, and pose landmarks. These extracted keypoints were then sequentially combined to form a representation of each gesture video. The resulting sequence of keypoints served as the input for training an LSTM-based model to capture spatial-temporal dependencies in sign movements. The trained model was subsequently utilized for real-time prediction, where gestures performed live by the user are detected and classified – this process is explained in detail in the later section.

A flow diagram of the proposed MP-GestLSTM model is depicted in Figure 1. The diagram represents the overall workflow, including setup, data preprocessing, model construction, training and testing, and real-time inference. Initially, video data were collected using a webcam and then processed by extracting frames for easier handling. Keypoints or landmarks are then extracted from these frames and stored as NumPy arrays. Landmark dictionaries were built for training and testing using landmark arrays retrieved from the videos. Class labels were assigned to the data, and landmark arrays were padded to ensure a uniform size. Class labels are categorized using label and one-hot encoding techniques. Subsequently, an LSTM model was constructed and compiled, followed by training using the prepared labels and features. The training process was monitored

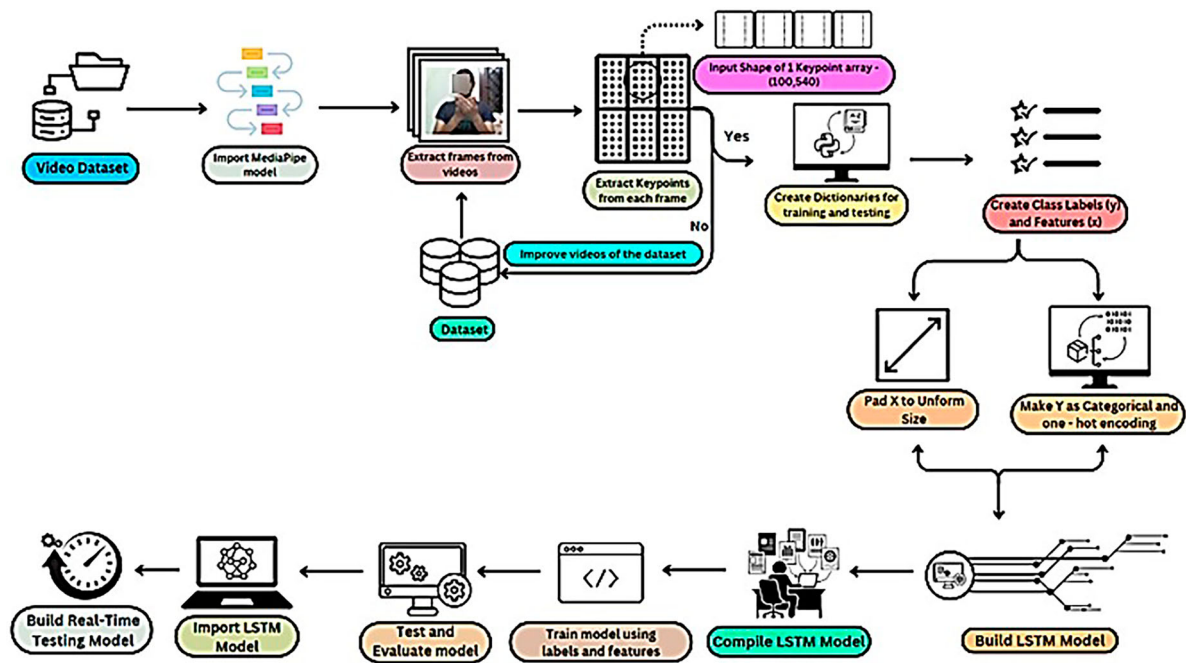


Figure 1. Schematic diagram of MP-GestLSTM model.

by plotting accuracy and loss graphs. Upon completion of training, the model was tested and evaluated using dedicated testing data. Additionally, a real-time testing application was developed to assess the performance of the model in live scenarios and support assistive communication applications.

Details of the MP-Gest dataset and libraries used to build and train the model are explained in section 3.1. Section 3.2 explains the data preprocessing steps, and Section 3.3 provides details of the training, testing, and evaluation of the MP-GestLSTM model.

3.1. MP-gest dataset creation and libraries used

Training models using existing datasets, such as WLASL (Li et al., 2020) wasn't efficient due to various factors. First, it contains a limited number of videos per class, restricting the diversity and robustness of the model training data. Second, the dataset suffers from issues related to illumination conditions, partial occlusions, different viewpoints, and variations in resolutions. These factors make it difficult to extract high-quality frames from videos. In addition, background artifacts further complicate the process of obtaining clean and informative frames for analysis thus hindering the development of accurate and reliable models for sign language recognition. Strategies to improve frame extraction methods, increase dataset diversity, and lessen the impact of ambient factors during data collection and preprocessing are needed to address these problems. To address this issue a proprietary dataset named MP-Gest dataset consisting of 800 videos of 20 word-level ASL classes ('all', 'before', 'black', 'black 'book', 'book', 'candy', 'chair', 'clothes', 'computer', 'cousin', 'deaf', 'drink', 'fine', 'fine' 'go', 'go', 'go', 'help' 'no', 'no' 'no' thin, 'thin', 'walk', 'who', 'who' 'year', 'year', 'yes') are collected by two signers using the webcam. This dataset collection approach is video-based and effectively captures dynamic hand movements. Hand movements are an extremely significant feature of sign language recognition. This study prioritized informed consent, anonymity, and data security to uphold ethical standards and protect participants' rights.

The MP-GestLSTM model imports a variety of dependencies to facilitate various aspects of its functionality. These include standard libraries, such as `os` for operating system operations, `gc` for garbage collection, `cv2` for image processing, `time` for time-related operations, `shutil` for file operations, `numpy` for numerical operations, `tqdm` for progress bars, `mediapipe` for pose estimation and keypoint detection, `TensorFlow` for machine learning tasks, and `Matplotlib`, `pyplot` for visualization, `LabelEncoder` from `scikit-learn` for label encoding, `ThreadPoolExecutor` for concurrent execution of tasks, `FileLink` for creating links to files in Jupyter

notebooks, and to categorical from Keras for one-hot encoding. Additional dependencies include pandas for data manipulation, paths for working with file paths, and clear_output for clearing the output in Jupyter notebooks. These dependencies collectively enable various functionalities such as data preprocessing, model training, visualization, and evaluation of the proposed model.

3.2. Data preprocessing of MP-gest dataset

In the initial stage of preprocessing, frames were extracted from the collected videos. The dataset organization follows a hierarchical structure designed to facilitate efficient storage and retrieval of video frames for sign language recognition research. A main folder named 'Frame' serves as the primary repository for the dataset. This primary folder was divided into twenty subfolders, each corresponding to a distinct action class relevant to sign language gestures. Each action class subfolder was subdivided into 40 additional subfolders labelled numerically from 1 to 40. These secondary subfolders contain individual frames extracted from specific videos, with each subfolder dedicated to a single video instance. This organization ensures systematic access to frame data allowing easy extraction of keypoints from the frames.

The algorithm for extracting keypoints is explained in Algorithm 1.

Algorithm 1 Extract Landmarks From Frames

Input: Extracted Frames

Output: Keypoints arrays extracted from Frames

1. We define a function `get_frame_landmarks(frame)` that takes a frame as input and returns the landmarks detected in the frame.
 2. Initialize an array of all landmarks with zeros of shape $(\text{HAND_NUM} * 2 + \text{POSE_NUM} + \text{FACE_NUM}, 3)$ to store landmarks.
 3. We define three nested functions, `get_hands(frame)`, `get_pose(frame)`, and `get_face(frame)`, to process the frame for hand landmarks, pose landmarks, and face landmarks, respectively.
 4. Inside the `get_hands(frame)` function:
 - 4.1. Process the frame using the Hands model to detect hand landmarks.
 - 4.2. If hand landmarks are detected (`results_hands.multi_hand_landmarks` is not empty):
 - 4.2.1. Iterate over the detected hand landmarks.
 - 4.2.2. Determine whether the hand is right or left based on `multi_handedness`.
 - 4.2.3. The coordinates (x, y, z) of each hand landmark are extracted and stored in all landmark arrays, separating the right-hand landmarks from the left-hand landmarks.
 5. Inside the `get_pose(frame)` function:
 - 5.1. Process the frame using the Pose model to detect pose landmarks.
 - 5.2. If pose landmarks are detected (`results_pose.pose_landmarks` is not empty):
 - 5.2.1. The coordinates (x, y, z) of each pose landmark are extracted and stored in all landmark arrays at the appropriate indices based on `POSE_NUM` and `filtered_pose`.
 6. Inside the `get_face(frame)` function:
 - 6.1. Process the frame using the FaceMesh model to detect face landmarks.
 - 6.2. If face landmarks are detected (`results_face.multi_face_landmarks` is not empty):
 - 6.2.1. The coordinates (x, y, z) of each face landmark are extracted and stored in all landmark arrays at the appropriate indices based on `FACE_NUM` and `filtered_face`.
 7. Use `ThreadPoolExecutor` with `max_workers = 3` to concurrently execute the three functions (`get_hands`, `get_pose`, and `get_face`) to improve efficiency.
 8. Return the flattened array of all landmarks containing the coordinates of all the detected landmarks in the frame.
-

The selected keypoints from the MediaPipe library for hand, pose, and face detection were utilized to extract landmarks from the video frames. For hand detection, 21×2 keypoints were selected, which corresponded to the landmarks representing the fingers and palm of each hand. These keypoints provide essential information regarding hand gestures and movements. In pose detection, six keypoints are chosen, representing key body joints, such as the shoulders, elbows, and wrists. These pose keypoints offer insights into body posture and movement patterns. Additionally, for face detection, a comprehensive set of 132 keypoints was employed, covering facial features such as the eyes, nose, mouth, and eyebrows. These facial landmarks enable analysis of facial expressions and gestures. Thus, 180 ($132 + 6 + 21 + 21$) keypoints were extracted from a frame. These specific sets of keypoints extracted from video frames give a detailed representation of hand, body, and facial movements which are used in various applications, such as gesture recognition and human-computer interaction.

The selected keypoints are as follows:

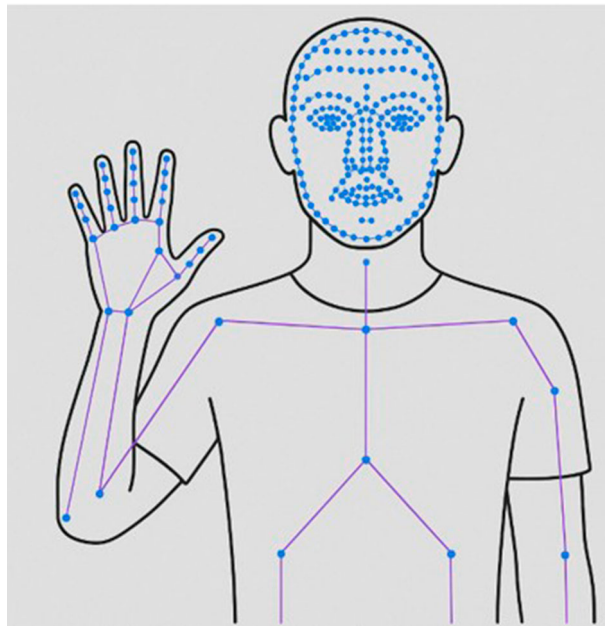


Figure 2. MP-GestLSTM model selected keypoints.

- filtered_hand = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
- filtered_pose = [11, 12, 13, 14, 15, 16]
- filtered_face = [0, 4, 7, 8, 10, 13, 14, 17, 21, 33, 37, 39, 40, 46, 52, 53, 54, 55, 58, 61, 63, 65, 66, 67, 70, 78, 80, 81, 82, 84, 87, 88, 91, 93, 95, 103, 105, 107, 109, 127, 132, 133, 136, 144, 145, 146, 148, 149, 150, 152, 153, 154, 155, 157, 158, 159, 160, 161, 162, 163, 172, 173, 176, 178, 181, 185, 191, 234, 246, 249, 251, 263, 267, 269, 270, 276, 282, 283, 284, 285, 288, 291, 293, 295, 296, 297, 300, 308, 310, 311, 312, 314, 317, 318, 321, 323, 324, 332, 334, 336, 338, 356, 361, 362, 365, 373, 374, 375, 377, 378, 379, 380, 381, 382, 384, 385, 386, 387, 388, 389, 390, 397, 398, 400, 402, 405, 409, 415, 454, 466, 468, 473]

These keypoints were carefully selected based on their functional relevance to sign language – the hand landmarks capture finger and palm movements essential for gestures, the pose landmarks track arm and shoulder positioning for spatial context, and the facial landmarks represent expressions and lip movements that add meaning. This selection is typically made through knowing that hand and finger movements carry the main meaning, arm positions give spatial context, and facial expressions add emotion or grammatical cues. The keypoints that were not selected primarily correspond to body parts such as the legs, torso, and less expressive facial regions, which contribute little to sign language interpretation. Since sign language communication relies mainly on hand gestures, arm movements, and facial expressions, these other landmarks add unnecessary data that can introduce noise and increase computational complexity without improving accuracy. Including them could also make the model slower and more prone to overfitting, as it would try to learn from irrelevant movements or static body parts. Therefore, excluding these non-essential keypoints helps the model stay focused on the most informative features, leading to faster, cleaner, and more accurate sign recognition. Figure 2 shows the selected keypoints.

Future work can include a comparative analysis of different landmark subsets to further enhance the proposed methodology. Although this study employed specific hand, pose, and facial keypoints (filtered_hand, filtered_pose, and filtered_face), evaluating alternative combinations could reveal the relative importance of each subset in improving recognition accuracy and computational efficiency. Such an analysis would provide deeper insights into feature selection and contribute to a more optimized and robust MP-GestLSTM framework for real-world sign language applications.

A primary folder termed 'Landmark' is created for the dataset. Within this folder, 20 distinct subfolders were allocated, each corresponding to a specific action class relevant to sign language gestures. Each action class subfolder contains a collection of 40 numpy arrays, each representing the keypoints extracted from all frames

of an individual video instance. The keypoint arrays encapsulate the spatial coordinates of landmarks detected within each frame, with each landmark defined by three dimensions (latitude, longitude, and depth), resulting in 540 keypoints per frame.

Furthermore, the dimensionality of the keypoint arrays varies across videos due to discrepancies in the number of frames extracted, denoted by the variable 'f'. Since the number of frames extracted differs from video to video, the keypoints arrays are padded after the creation of class feature (X_{train} and X_{test}) and class label (y_{train} and y_{test}) and thus the resulting shape of one keypoint array of a video is 100×540 . The padding function is explained in Algorithm 2.

Additionally, in the dataset preparation phase, two dictionaries, `landmarks_TEST.npz` and `landmarks_TRAIN.npz`, are meticulously crafted to organize and store keypoint landmarks essential for subsequent training and testing processes in sign language recognition research. Each dictionary is structured with keys denoting action classes, labelled as `actionClass_i`, where 'actionClass' signifies the specific sign language gesture category, and 'i' ranges from 1 to 40, representing the video number. The values corresponding to these keys encapsulate the landmarks extracted from the respective videos, providing comprehensive spatial landmark data that are crucial for model training and evaluation. Subsequently, these dictionary datasets served as the foundational components for generating training and testing datasets, denoted as X_{train} , y_{train} , X_{test} , and y_{test} , facilitating the delineation of distinct datasets for model training and evaluation purposes.

Algorithm 2 Padding function

Input: Class Features (X) and Class Labels (y)

Output: Padded X With Uniform Shape

1. We define a function padding ($X, Y, \text{length} = \text{None}, \text{pad} = 0$) that pads the sequences in X to a specified length.
 2. If the length parameter is not provided, it is set as the maximum length of the sequences in X .
 3. Initialize an empty list X_{padded} to store the padded sequences.
 4. Iterate over each sequence x in X :
 - 4.1. If the length of x is greater than the specified length:
 - 4.1.1. Truncate x to the specified length and append it to X_{padded} .
 - 4.2. Otherwise (if the length of x is less than or equal to the specified length):
 - 4.2.1. Calculate the number of elements to the pad (pad_length) by subtracting the length of x from the specified length.
 - 4.2.2. Pad x with zeros (or the specified pad value) to make its length equal to the specified length using `np.pad`.
 - 4.2.3. Append the padded sequence to X_{padded} .
 5. Convert the list X_{padded} to a numpy array.
 6. Return the padded sequences X_{padded} along with the labels Y .
-

3.3. MP-GestLSTM model

The LSTM model for this study was designed for sequence classification tasks, particularly suited for processing temporal data such as time series or sequences of features, as in this case. It comprises four main components: a masking layer, LSTM layers, a flattened layer, and a dense layer. The algorithm for the proposed MP-GestLSTM model is presented in Algorithm 3. The model begins with a masking layer with a mask value of 100. The input shape of the LSTM model is (100, 540). This layer is typically used to handle variable-length sequences by masking any padded value. Following the Masking layer, two LSTM layers were used. LSTM (Long Short-Term Memory) units are a type of recurrent neural network (RNN) architecture designed to capture long-term dependencies in sequential data. The first LSTM layer has 64 units and returns sequences, meaning that it processes input sequences and produces output sequences of the same length. Similarly, the second LSTM layer with 32 units returned the sequences.

A flattened layer is then added to the model after the LSTM layers. The output from the LSTM layers is flattened by this layer into a one-dimensional array so that it may be fed into the dense layer that follows. The model ended with a dense layer of 20 units and a linear activation function. This layer serves as the output layer to produce the final classification predictions. The linear activation function implies that the output values are directly proportional to the input values, without any nonlinear transformation.

Algorithm 3 MP-GestLSTM model**Input:** Class Features (X) and Class Labels (y)**Output:** Efficient Sign Language Detection Model.

1. Create a Sequential model instance.
2. A masking layer was added to the model with a mask value of -100 and input shape (100, 540).
3. Add an LSTM layer to the model with 64 units and return sequences.
4. Add another LSTM layer to the model with 32 units and return sequences.
5. Add a Flatten layer to the model.
6. A dense layer with 20 units and a linear activation function was added to the model.
7. Create a directory called 'checkpoints' to store model checkpoints if they do not exist.
8. We define a ModelCheckpoint callback to save the model weights after each epoch, ensuring that only the best weights are saved based on validation accuracy.
9. We define a learning rate scheduler using ExponentialDecay with an initial learning rate of 0.001, decay steps of 1000, and decay rate of 0.9.
10. We define an EarlyStopping callback to monitor the validation accuracy and stop training if there is no improvement after 50 epochs, restoring the best weights.
11. The model is compiled using cosine similarity as the loss function. The model metrics include Adam optimizer with the defined learning rate scheduler and accuracy.
12. The model was fitted to the training data (X_train, y_train_one_hot) for 100 epochs with a batch size of 64 and was validated with the validation data (X_val, y_val_one_hot).
13. Callbacks defined during training was used to save the best model weights and stop training early if necessary.

The model also incorporates a few callbacks to monitor and optimize the training process, including an exponential decay learning rate scheduler to adaptively modify the learning rate during training, EarlyStopping to prevent overfitting by stopping training when the validation accuracy stops improving, and ModelCheckpoint to save the optimal model weights. A cosine similarity loss function, an Adam optimizer with a learning rate scheduler, and accuracy were used as the evaluation metric to create the model. The system's webcam is used to provide input to the real-time testing module which is explained section 4.2.

4. Results and discussion

The MP-GestLSTM model architecture is an optimal solution for sequence-prediction tasks. The input sequences have a shape of (100, 540). The architecture consists of a masking layer followed by two layers of Long Short-Term Memory (LSTM) units, a flattened layer, and a dense layer with 20 units and a linear activation function. Several hyperparameters were tuned to optimize the training process. The Masking layer was used to handle variable-length sequences with a mask value of -100 . The Adam optimizer was chosen with an initial learning rate of 0.001.

Additionally, an exponential decay learning rate scheduler was implemented to dynamically adjust the learning rate during training, decaying it by a factor of 0.9 every 1000 steps. Two callbacks were employed to monitor the performance of the MP-GestLSTM model and prevent overfitting. The ModelCheckpoint callback saves the model weights after each epoch if the validation accuracy improves, thereby ensuring that the best-performing model is retained. The EarlyStopping callback monitored the validation accuracy and halted training if there was no improvement for 50 consecutive epochs, restoring the weights of the best-performing model.

4.1. Performance of MP-GestLSTM

The MP-GestLSTM model obtained a training accuracy of 0.9984 and a validation accuracy of 0.9375. The MP-GestLSTM model achieved a testing accuracy of 0.9438. The loss and accuracy graphs of MP-GestLSTM are shown in Figure 3.

Figure 4 shows the confusion matrix of MP-GestLSTM. Table 2 provides a comparison between the existing method and the proposed MP-GestLSTM. Table 3 presents the classification results for the proposed MP-GestLSTM.

The proposed MP-GestLSTM model demonstrates superior performance compared to existing methods across multiple sign language datasets. As presented in Table 2, the model achieved an accuracy of 94.38%, surpassing earlier approaches such as the HHDS multi-stream 3D CNN (90.00%) and various RGB + DM-based HHDS models, which reported accuracies between 50.00% and 94.00%. The results highlight the model's

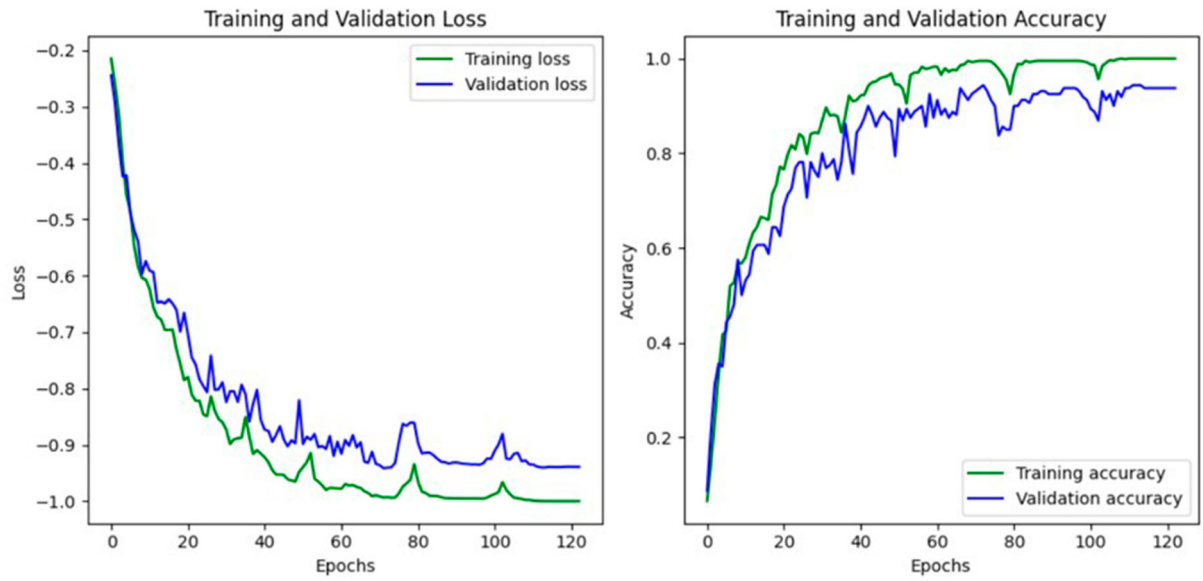


Figure 3. MP-GestLSTM model accuracy and loss graph.

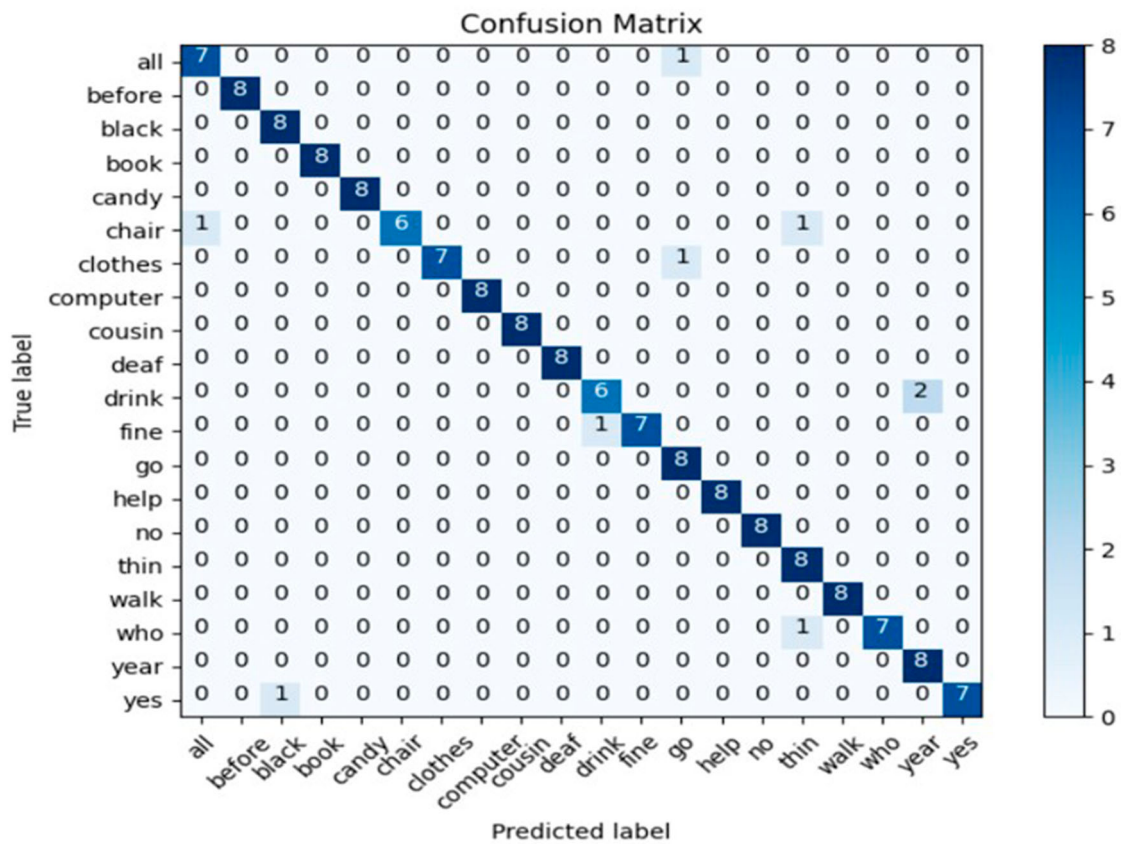


Figure 4. Confusion matrix of MP-GestLSTM model.

robustness and adaptability across diverse datasets, including MINDS-Libras, INCLUDE-50, Korean Sign Language, and LIBRAS-UFOP. The enhanced accuracy indicates that MP-GestLSTM effectively captures spatial and temporal dependencies in gesture sequences, leading to more reliable recognition performance under varying conditions and class complexities.

Table 2. Comparative analysis of existing literature and MP-GestLSTM.

Model	Accuracy
HHDS (hands + head + distance and speed maps) Multi-stream 3D CNN (De Castro et al., 2023)	90.00
RGB + DM (Depth Maps) HHDS (De Castro et al., 2023)	91.00
RGB + DM HHDS (De Castro et al., 2023)	94.00
RGB + DM HHDS (De Castro et al., 2023)	50.00
RGB + DM HHDS (De Castro et al., 2023)	74.00
Masking future operation with full self-attention ViT framework (Du et al., 2022)	Top1 – 80.72
Masking future operation with full self-attention ViT framework (Du et al., 2022)	Top1 – 70.14
Masking future operation with full self-attention ViT framework (Du et al., 2022)	Top1 – 57.13
Proposed MP-GestLSTM	94.38

Table 3. Classification report of MP-GestLSTM.

	Precision	Recall	F1-score	Support
all	0.88	0.88	0.88	8
before	1.00	1.00	1.00	8
black	0.89	1.00	0.94	8
book	1.00	1.00	1.00	8
candy	1.00	1.00	1.00	8
chair	1.00	0.75	0.86	8
clothes	1.00	0.88	0.93	8
computer	1.00	1.00	1.00	8
cousin	1.00	1.00	1.00	8
deaf	1.00	1.00	1.00	8
drink	0.86	0.75	0.80	8
fine	1.00	0.88	0.93	8
go	0.80	1.00	0.89	8
help	1.00	1.00	1.00	8
no	1.00	1.00	1.00	8
thin	0.80	1.00	0.89	8
walk	1.00	1.00	1.00	8
who	1.00	0.88	0.93	8
year	0.80	1.00	0.89	8
yes	1.00	0.88	0.93	8
accuracy			0.94	160
macro avg	0.95	0.94	0.94	160
weighted avg	0.95	0.94	0.94	160

4.2. Performance of real-time application

To implement real-time gesture recognition, the function `mediapipe_detection` (image, model) is defined to process the input frame using a MediaPipe holistic model, returning both the processed image and detected keypoints. Subsequently, two empty lists, sequence and sentence, were initialized to store gesture keypoint sequences and predicted actions, respectively, with a threshold value set to determine the prediction confidence. The MediaPipe holistic model was then set up, configuring the minimum detection and tracking confidence thresholds. Following this, a video capture device, such as a webcam, is initialized. Within a loop that continues while the video feed is active, frames are read from the feed, and detections are made using the MediaPipe holistic model. Keypoints were extracted from the frame, appended to the sequence list, and limited to the last 100 elements. If the sequence contains 100 keypoints, the MP-GestLSTM model predicts the action label, which is printed and used to update the sentence constrained to the last five actions. Additionally, a rectangle and text displaying the current sentence are drawn on the image, which is then displayed on the screen using OpenCV. The loop terminates upon pressing the 'q' key. Finally, the video capture device was released, and all OpenCV windows were closed to ensure the proper termination of the programme.

To validate the system's real-time performance, the code was executed on a live webcam feed using parallel processing via `ThreadPoolExecutor` to simultaneously extract face, hand, and pose keypoints, thereby minimizing latency. Frame-by-frame inference was performed with MediaPipe and the trained LSTM model, achieving a consistent frame rate of approximately 25–30 FPS on a standard CPU, confirming real-time responsiveness.

The entire implementation was developed as a simple Python-based interface using libraries such as OpenCV, MediaPipe, NumPy, and TensorFlow/Keras, ensuring compatibility and ease of deployment. Testing was carried out on a mid-range laptop equipped with an Intel i5 processor, 8GB RAM, and integrated graphics, without the need for GPU acceleration. This demonstrates that the system operates efficiently on general-purpose hardware, supporting the claim of real-time gesture detection and making it suitable for edge deployment and user-interactive applications. Algorithm 4 explains the implemented methodology.

This comprehensive approach provides a framework for real-time gesture recognition and interpretation, thereby facilitating interactive applications and user communication. The results of real-time predictions are shown in Figure 5. The interface for the real-time application is shown in Figure 6.

Algorithm 4 Real-Time Gesture Detection using MP-GestLSTM

Input: Live video stream captured from webcam

Output: Real-time predicted gesture labels displayed on screen

1. Initialize MediaPipe models: Create instances of `mp.solutions.hands.Hands()`, `mp.solutions.pose.Pose()`, and `mp.solutions.face_mesh.FaceMesh(refine_landmarks = True)` to extract hand, pose, and face landmarks.
 2. Define the `get_frame_landmarks(frame)` function:
 - Initialize an array `all_landmarks` of zeros with dimensions $(\text{HAND_NUM} * 2 + \text{POSE_NUM} + \text{FACE_NUM}, 3)$.
 - Define `get_hands(frame)` to detect and extract right and left hand landmarks.
 - Define `get_pose(frame)` to detect and extract selected pose landmarks using the `filtered_pose` indices.
 - Define `get_face(frame)` to detect and extract selected face landmarks using the `filtered_face` indices.
 - Use a `ThreadPoolExecutor` with 3 workers to run hand, pose, and face detection in parallel.
 - Flatten and return the concatenated landmark array.
 3. Define the `mediapipe_detection(image, model)` function:
 - Convert the frame colour from BGR to RGB.
 - Set the image as non-writeable and process it using the holistic model to obtain detection results.
 - Convert the image colour back to BGR and return both the processed image and the results.
 4. Initialize detection variables: Create empty lists sequence and sentence, and set a threshold value (e.g. 0.8) to control confidence-based action updates.
 5. Initialize webcam and holistic model:
 - Use `cv2.VideoCapture(0)` to access the webcam.
 - Load the holistic model with `min_detection_confidence = 0.5` and `min_tracking_confidence = 0.5`.
 6. Begin real-time detection loop:
 - Continuously read frames from the webcam feed while it is active.
 - Perform detection on each frame using the `mediapipe_detection()` function.
 - Extract hand, pose, and face keypoints using the `get_frame_landmarks()` function.
 - Append the extracted keypoints to the sequence list, retaining only the most recent 100 frames.
 7. Action prediction:
 - When 100 frames are accumulated, reshape the sequence and pass it to the trained MP-GestLSTM model for prediction.
 - Identify the predicted action label with the highest probability.
 - If the prediction confidence exceeds the threshold, update the sentence list with the current action.
 8. Display results:
 - Draw a rectangular overlay on the video frame.
 - Display the latest recognized gesture (sentence) using OpenCV's `putText()` function.
 - Show the annotated frame in a window titled 'Sign Language Classification'.
 9. Terminate programme:
 - Exit the loop when the user presses the 'q' key.
 - Release the webcam using `cap.release()` and close all OpenCV windows with `cv2.destroyAllWindows()`.
-

5. Conclusion and future work

The main goal of this work was to develop an accurate and efficient real-time sign language translation model using MP-Gest keypoint extraction and an LSTM-based architecture. The proposed MP-GestLSTM model achieved a training accuracy of 0.9984, validation accuracy of 0.9375, and testing accuracy of 0.9438, which demonstrates a notable improvement over existing approaches that often struggled to reach 70% accuracy with more complex networks. This work contributes by designing a simplified, yet highly effective architecture tailored for video-based gesture recognition, along with a structured preprocessing pipeline that handles variable-length sequences and extracts the most relevant spatial-temporal features for improved gesture interpretation.

For the real time application, the system was implemented as a Python-based real-time interface that translates sign gestures into text. Its lightweight nature makes it suitable for use on general-purpose computers and



Figure 5. Real-time predictions of MP-GestLSTM model.

potential integration into low-resource or assistive communication devices. The model's simplicity also supports scalability, providing a flexible base for further development of interactive and accessible sign language tools.

Despite its strong performance, the work has certain limitations. The dataset used involved only two signers and a limited vocabulary, which constrains generalization across users and contexts. Varying backgrounds,

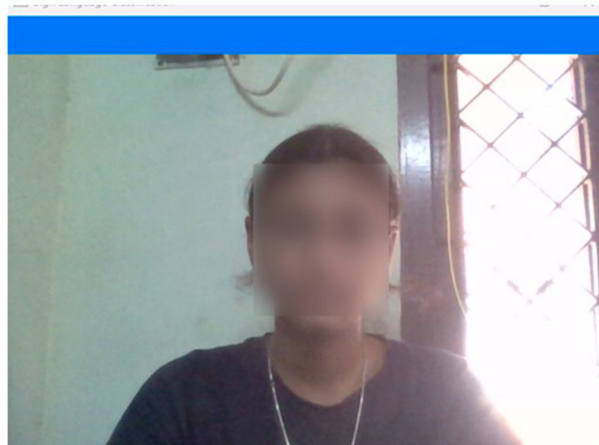


Figure 6. Interface of real-time application.

lighting conditions and multi-individual scenarios can affect the efficiency of keypoint detection. The current padding function standardizes video lengths by adding zeros. This approach may introduce small discrepancies in motion continuity. This issue could be reduced in future work by employing an adaptive padding approach that uses interpolated values from neighbouring frames.

Expanding the dataset would generalize the model better across users. Typically, more signers and a wider vocabulary would be required for future development. Natural language translation and complex environments like continuous sign sequences can be easily recognized by incorporating multi-modal data, such as depth or audio information.

The model accuracy and computational performance can be improved with a comparative analysis approach of landmark subsets to understand the contribution of each body region.

Incorporating these changes would make the model a more robust and accessible sign language translation system and improve communication for individuals with hearing impairment.

Author contributions statement

Varshini T.S: Conception and design, drafting of the paper Rukmani P: Revising it critically for intellectual content.

All authors have read and approved the final version of the manuscript.

Consent

All studies involving human participants were conducted in accordance with the principles outlined in the Declaration of Helsinki.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this manuscript, the authors used ChatGPT 4 (OpenAI) to improve readability and language. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

Disclosure statement

No potential conflict of interest was reported by the author(s).

Data availability statement

The data that support the findings of this study are Openly available in <https://data.mendeley.com/datasets/3cpyspxy4n/1>.

References

- Boháček, M., & Hruží, M. (2022, January 4–8). Sign pose-based transformer for word-level sign language recognition. In *2022 IEEE/CVF Winter Conference on Applications of Computer Vision Workshops (WACVW)*, Waikoloa, HI, USA (pp. 182–191). <https://doi.org/10.1109/WACVW54805.2022.00024>
- De Castro, G. Z., Guerra, R. R., & Guimarães, F. G. (2023). Automatic translation of sign language with multi-stream 3D CNN and generation of artificial depth maps. *Expert Systems with Applications*, 215, 119394. ISSN 0957-4174. <https://doi.org/10.1016/j.eswa.2022.119394>.
- De Coster, M., Van Herreweghe, M., & Dambre, J. (2020, May 11–16). Sign language recognition with transformer networks. In *12th international conference on language resources and evaluation* (pp. 6018–6024). European Language Resources Association (ELRA).
- Du, Y., Xie, P., Wang, M., Hu, X., Zhao, Z., & Liu, J. (2022). Full transformer network with masking future for word-level sign language recognition. *Neurocomputing*, 500, 115–123. ISSN 0925-2312. <https://doi.org/10.1016/j.neucom.2022.05.051>
- Elsayed, E. K., & Fathy, D. R. (2020). Sign language semantic translation system using ontology and deep learning. *International Journal of Advanced Computer Science and Applications*, 11(1), 141–147. <https://doi.org/10.14569/IJACSA.2020.0110118>
- He, S. (2019, October 17–19). Research of a sign language translation system based on deep learning. In *2019 international conference on artificial intelligence and advanced manufacturing (AIAM)*, Dublin, Ireland (pp. 392–396). <https://doi.org/10.1109/AIAM48774.2019.00083>
- Kasapbaşı, A., Elbushra, A. E. A., Al-Hardane, O., & Yilmaz, A. (2022). DeepASLR: A CNN based human computer interface for American Sign Language recognition for hearing-impaired individuals. *Computer Methods and Programs in Biomedicine Update*, 2, 100048. ISSN 2666-9900. <https://doi.org/10.1016/j.cmpbup.2021.100048>.
- Kothadiya, D. R., Bhatt, C. M., Saba, T., Rehman, A., & Bahaj, S. A. (2023). SIGNFORMER: DeepVision transformer for sign language recognition. *IEEE Access*, 11, 4730–4739. <https://doi.org/10.1109/ACCESS.2022.3231130>
- Li, D., Opazo, C. R., Yu, X., & Li, H. (2020, March 1–5). Word-level deep sign language recognition from video: A New large-scale dataset and methods comparison. In *2020 IEEE Winter Conference on Applications of Computer Vision (WACV)* (pp. 1448–1458). <https://doi.org/10.1109/WACV45572.2020.9093512>
- Orbay, A., & Akarun, L. (2020, November 16–20). Neural sign language translation by learning tokenization. In *2020 15th IEEE international conference on automatic face and gesture recognition (FG 2020)*, Buenos Aires, Argentina (pp. 222–228). <https://doi.org/10.1109/FG47880.2020.00002>
- Sharma, S., & Singh, S. (2021). Vision-based hand gesture recognition using deep learning for the interpretation of sign language. *Expert Systems with Applications*, 182, 115657. ISSN 0957-4174. <https://doi.org/10.1016/j.eswa.2021.115657>
- Sharma, S., & Singh, S. (2023). ISL recognition system using integrated mobile-net and transfer learning method. *Expert Systems with Applications*, 221, 119772. ISSN 0957-4174. <https://doi.org/10.1016/j.eswa.2023.119772>
- Sign Language Translation Using Deep Convolutional Neural Networks. (2020, February 29). KSII transactions on internet and information systems. *Korean Society for Internet Information (KSII)*. <https://doi.org/10.3837/tiis.2020.02.009>
- Sreemathy, R., Turuk, M. P., Chaudhary, S., Lavate, K., Ushire, A., & Khurana, S. (2023). Continuous word level sign language recognition using an expert system based on machine learning. *International Journal of Cognitive Computing in Engineering*, 4, 170–178. ISSN 2666-3074. <https://doi.org/10.1016/j.ijcce.2023.04.002>.
- Stellin, R., Rukmani, P., Anbarasi, L. J., & Narayanan, S. (2022). HandGCNN model for gesture recognition based voice assistance. *Multimedia Tools and Applications*, 81(29), 42353–42369. <https://doi.org/10.1007/s11042-022-13497-5>
- Sunanda Das, M., Imtiaz, S., Neom, N. H., Siddique, N., & Wang, H. (2023). A hybrid approach for bangla sign language recognition using deep transfer learning model with random forest classifier. *Expert Systems with Applications*, 213(Part B), 118914. ISSN 0957-4174. <https://doi.org/10.1016/j.eswa.2022.118914>
- Svendsen, B., & Kadry, S. (2024). A dataset for recognition of Norwegian sign language. *International Journal of Mathematics, Statistics, and Computer Science*, 2, 1–3. <https://doi.org/10.59543/ijmscs.v2i.8049>
- Tolentino, L. K. S., Juan, R. S., Thio-ac, A. C., Pamahoy, M. A. B., Forteza, J. R. R., & Garcia, X. J. O. (2019). Static sign language recognition using deep learning. *International Journal of Machine Learning and Computing*, 9(6), 821–827. <https://doi.org/10.18178/ijmlc.2019.9.6.879>
- Zhou, H., Zhou, W., Qi, W., Pu, J., & Li, H. (2021, June 19–25). Improving sign language translation with monolingual data by sign back-translation. In *2021 IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, Nashville, TN, USA (pp. 1316–1325). <https://doi.org/10.1109/CVPR46437.2021.00137>

Appendix

Summary of MP-GestLSTM Model is shown in Figure A1.

Model: "sequential"

Layer (type)	Output Shape	Param #
masking (Masking)	(None, 100, 540)	0
lstm (LSTM)	(None, 100, 64)	154880
lstm_1 (LSTM)	(None, 100, 32)	12416
flatten (Flatten)	(None, 3200)	0
dense (Dense)	(None, 20)	64020

=====
 Total params: 231316 (903.58 KB)
 Trainable params: 231316 (903.58 KB)
 Non-trainable params: 0 (0.00 Byte)

Figure A1. Summary of MP-GestLSTM model.